

CYBERSECURITY SPRINT 3

PrevioPLS · DevSecOps

Challenge FIAP 2026 · Ford — Desafio 02 (VIN Share)

Giovanne Charelli Zaniboni Silva

RM 556223

Gustavo Oliveira de Moura

RM 555827

Lynn Bueno Rosa

RM 551102

26 de setembro de 2026

00 Contexto e escopo

01 Pipeline DevSecOps integrado (peso 3,0)

- 1.1 Desenho do pipeline
- 1.2 Etapas, ferramentas e o risco que cada uma reduz
- 1.3 Como o pipeline roda no projeto Ford inteiro
- 1.4 Evidência de execução: antes × depois
- 1.5 O pipeline nas outras frentes da Sprint 3
- 1.6 Rotina contínua ligada ao pipeline

02 Segurança em código e infraestrutura (peso 2,5)

- 2.1 Correções reais no código (antes → depois)
- 2.2 Criptografia local (dados em repouso)
- 2.3 Hardening da API
- 2.4 Controle de acesso por perfil
- 2.5 Segurança MQTT/TLS para IoT
- 2.6 IaC Security (Dockerfile, Compose, Kubernetes)

03 Observabilidade, monitoramento e resposta (peso 2,0)

- 3.1 Arquitetura de observabilidade
- 3.2 Logs estruturados
- 3.3 Métricas e alertas por domínio
- 3.4 Dashboards (prints da execução)
- 3.5 Plano de resposta a incidentes

04 Compliance, riscos e segurança contínua (peso 2,5)

- 4.1 Revisão final dos riscos — STRIDE por componente
- 4.2 Mapeamento OWASP ASVS 4.0.3
- 4.3 OWASP API Security Top 10 (2023) e Mobile Top 10 (2024)
- 4.4 LGPD — dados pessoais, telemetria e localização
- 4.5 Plano de segurança contínua
- 4.6 Checklist de conformidade
- 4.7 Riscos residuais e roadmap

— Anexo A — histórico de commits

— Anexo B — como reproduzir

0. Contexto e escopo

Tudo o que está aqui foi **executado** na stack deste repositório em 26/09/2026. Os números vêm dos arquivos em [docs/evidencias/](#), e cada correção de código tem um commit próprio no histórico git (lista completa no Anexo A).

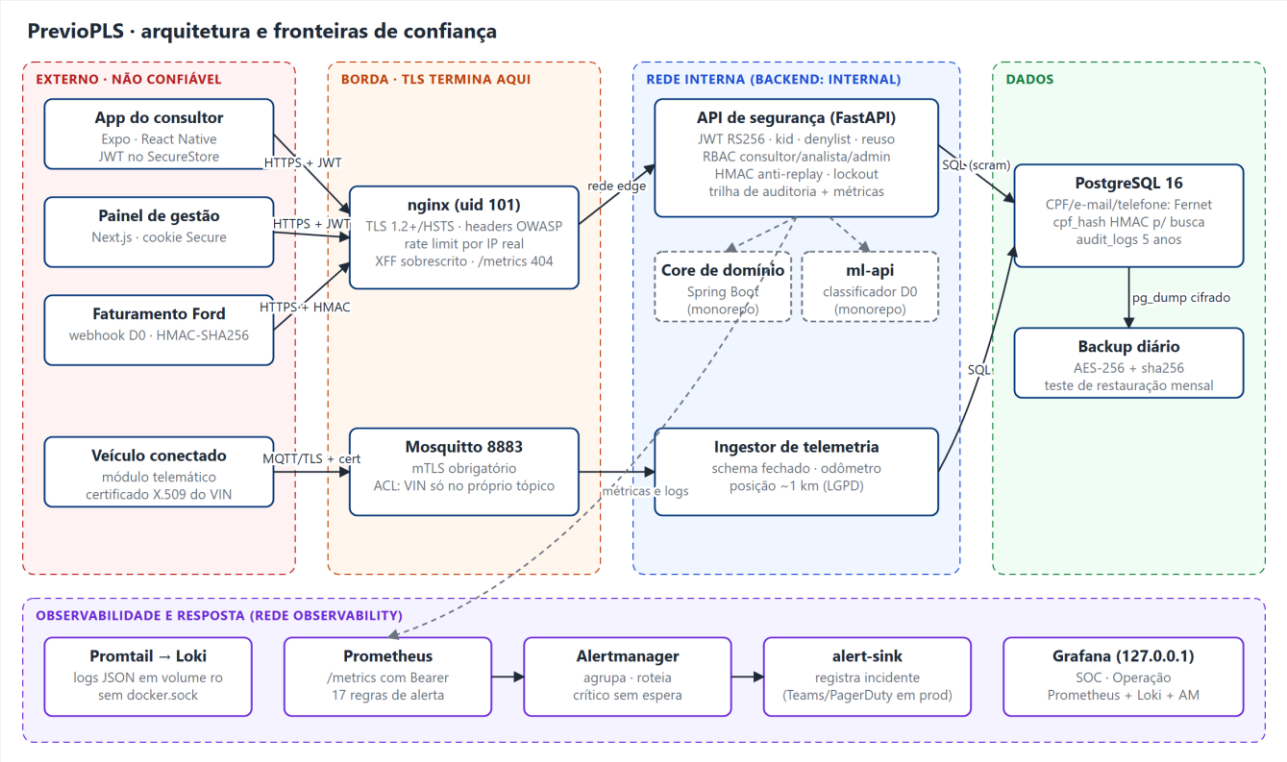
Como o documento está organizado

#	Atividade	Peso	Seção
1	Pipeline DevSecOps integrado	3,0	§1
2	Segurança em código e infraestrutura	2,5	§2
3	Observabilidade, monitoramento e resposta	2,0	§3
4	Compliance, riscos e segurança contínua	2,5	§4

O PrevioPLS classifica o comprador de um Ford no momento da compra (D0) em quatro perfis (fiel, econômico, esquecido, abandono) e gera leads priorizados para o consultor da concessionária agir antes que o cliente deixe a rede oficial. A solução completa tem API, app mobile do consultor, painel web, classificador de ML e, nesta sprint, **telemetria de veículo conectado** (odômetro e códigos de falha OBD-II), que alimenta leads de revisão proativa.

Este repositório concentra a camada de segurança: a **API de segurança** (FastAPI, borda LGPD do monorepo PrevioPLS), o **ingestor de telemetria MQTT**, a infraestrutura como código e o pipeline. A API de domínio (Java/Spring Boot, [Sprint3-SOA](#)), o app do consultor (Expo, [Sprint3-mobile](#)) e o modelo de evasão (XGBoost, [Sprint3-IA-ML](#)) são entregas das outras disciplinas desta sprint e entram aqui pelo modelo de ameaças, pelo

mapeamento OWASP e pelos estágios 1 a 3 do pipeline rodados no código delas (\$1.5). O painel web (Next.js) vive no monorepo.



Arquitetura e fronteiras de confiança

Zona	Componentes	O que protege a fronteira
Externo (não confiável)	app do consultor, painel, faturamento Ford, veículo	nada é confiável: tudo autentica
Borda	nginx sem root, Mosquitto 8883	TLS 1.2+, rate limit por IP real, mTLS e ACL por VIN
Rede interna	API, ingestor, core, ml-api	JWT RS256 + RBAC, HMAC, validação de schema
Dados	PostgreSQL, backup	PII cifrada (Fernet), rede internal sem rota pra fora, backup AES-256
Observabilidade	Prometheus, Alertmanager, Loki, Grafana	só em 127.0.0.1, /metrics com Bearer

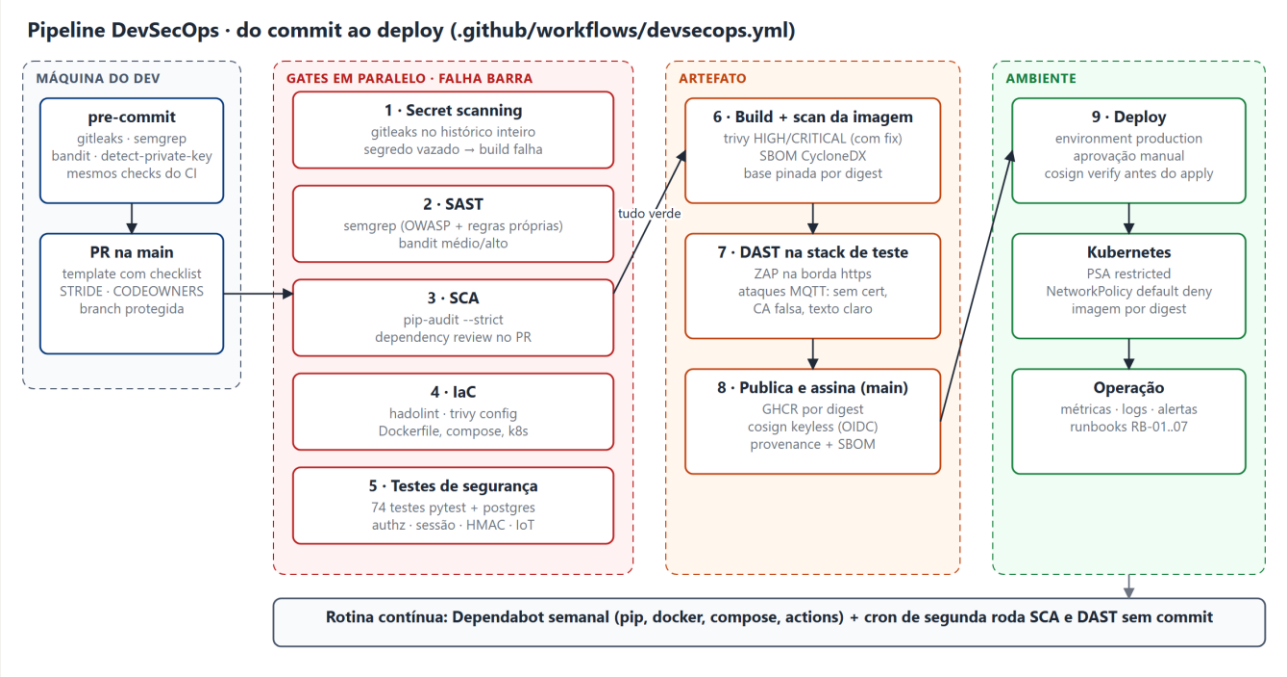
Perfis de acesso. A rubrica cita Brigadista, Gestor e Administrador como exemplo. No PrevioPLS os perfis equivalentes são:

Exemplo da rubrica	Perfil no PrevioPLS	Quem é	Pode
Brigadista (linha de frente)	Consultor	vendedor/pós-venda da concessionária (app)	ver e atualizar leads
Gestor	Analista	gestor de pós-venda / BI (painel)	ler leads, dashboards e trilha de auditoria; não altera nada
Administrador	Admin	responsável regional / segurança	cadastrar cliente, auditar, derrubar sessões
—	Veículo (dispositivo IoT)	módulo telemático, identificado pelo certificado X.509 do VIN	publicar só a própria telemetria
—	Integração de faturamento	sistema Ford que envia a venda D0	<code>POST /v1/clientes</code> assinado com HMAC

1. Pipeline DevSecOps integrado (peso 3,0)

1.1. Desenho do pipeline

Arquivo: `.github/workflows/devsecops.yml` (GitHub Actions).



Pipeline DevSecOps

O pipeline roda em todo push para a `main`, em todo pull request, **toda segunda às 06:00 UTC** (sem commit: CVE nova aparece sem o código mudar) e sob demanda. Os cinco primeiros estágios rodam em paralelo e **qualquer um vermelho impede o build da imagem**. Publicação e deploy só acontecem na `main`, e o deploy exige aprovação manual no environment `production`.

1.2. Etapas, ferramentas e o risco que cada uma reduz

#	Etapa	Ferramenta	Gate (quebra o build quando...)	Risco que reduz (STRIDE)
1	Secret scanning	GitLeaks no histórico inteiro (<code>fetch-depth: 0</code>), regras padrão + regras do projeto (<code>.gitLeaks.toml</code> : chave Fernet, segredo HMAC, pepper, PEM)	qualquer segredo encontrado	I vazamento de credencial pelo repositório; segredo removido em commit posterior continua no histórico e é achado
2	SAST	Semgrep (p/python, p/owasp-top-ten, p/jwt, p/secrets) + regras próprias em <code>.semgrep/previopls.yml</code> ; Bandit	achado ERROR no semgrep; bandit médio/alto	E rota sem RBAC; S JWT decodificado sem <code>algorithms</code> ; I PII em log; T TLS sem verificação; <code>--forwarded-allow-ips=*</code>
3	SCA	pip-audit --strict ; dependency-review no PR (CVE alta ou licença GPL/AGPL); Dependabot semanal	qualquer CVE conhecida numa dependência	T/E/D exploração de biblioteca vulnerável (supply chain)
4	IaC	Hadolint (Dockerfile) e Trivy config (Dockerfile e Kubernetes)	warning do hadolint; misconfig HIGH/CRITICAL	E container root, privilegiado, sem limite, sem NetworkPolicy
5	Testes de segurança	pytest contra Postgres real (service container): 75 testes	qualquer falha	regressão de controle (lockout, RBAC, sessão, HMAC, IoT)

#	Etapa	Ferramenta	Gate (quebra o build quando...)	Risco que reduz (STRIDE)
6	Imagem	Trivy image (HIGH/CRITICAL com correção) + SBOM CycloneDX	CVE HIGH/CRITICAL corrigível	T/E pacote vulnerável do SO ou do Python dentro da imagem
7	DAST	OWASP ZAP na borda HTTPS + ataques MQTT (sem certificado, CA falsa, texto claro) na stack de teste	alerta FAIL do ZAP (<code>.zap/rules.tsv</code>); broker aceitando conexão indevida	S/I falha que só aparece em execução (header, TLS, mTLS)
8	Publicação	GHCR por digest , provenance + SBOM, assinatura cosign keyless (OIDC do GitHub)	—	T imagem trocada no registry
9	Deploy	<code>cosign verify</code> do digest antes do <code>kubect1 apply</code> , environment com revisores	assinatura inválida	T/E subir artefato que não passou pelo pipeline

Proteções do próprio pipeline (supply chain de CI):

- todas as actions **pinadas por SHA** (tag é mutável); o Dependabot atualiza os SHAs;
- `permissions: contents: read` no topo e permissão extra só no job que precisa (`security-events`, `packages`, `id-token`);
- **CODEOWNERS** exige revisão de segurança em `.github/`, `app/core/`, `infra/`, `nginx/` e `iot/`;
- PR de fork não recebe segredo (padrão do GitHub) e o deploy nunca roda em `pull_request`;
- **shift-left**: `.pre-commit-config.yaml` roda gitleaks, semgrep e bandit na máquina do dev antes do commit sair; o template de PR traz checklist STRIDE.

1.3. Como o pipeline roda no projeto Ford inteiro

O mesmo desenho cobre o monorepo PrevioPLS com uma matriz por serviço; o que muda é a ferramenta de cada linguagem:

Serviço	SAST	SCA	Build/Image m	Específico
API de segurança / gateway (Python)	semgrep + bandit	pip-audit	trivy image	regras <code>.semgrep/</code> deste repo
API SOA (Java 21 / Spring Boot, <code>Sprint3-SOA</code>)	semgrep <code>p/java</code> + SpotBugs/FindSecBugs	OSV / <code>trivy fs</code> no <code>pom.xml</code> ; Dependabot <code>maven</code>	trivy image	testes de RBAC do Spring Security
IA/ML (Python + XGBoost, <code>Sprint3-IA-ML</code>)	semgrep + bandit + regra <code>modelo-carregado-sem-conferir-hash</code>	pip-audit com versões pinadas	trivy image	SHA-256 do <code>modelo_evasao.joblib</code> conferido antes do load (joblib executa código ao carregar)
Painel (Next.js)	semgrep <code>p/typescript</code> , <code>p/react</code>	<code>npm audit --omit=dev</code> / OSV-Scanner	trivy image	ZAP full scan no staging
App do consultor (Expo, <code>Sprint3-mobile</code>)	semgrep <code>p/typescript</code> , <code>p/react</code>	<code>npm audit</code> / OSV-Scanner	EAS Build → APK	MobSF estático no APK gerado (permissões, <code>allowBackup</code> , cleartext, segredos no bundle)
Infra	—	—	—	hadolint + trivy config + <code>docker compose config</code>
Telemetria (Mosquitto)	—	—	—	ataques MQTT do estágio 7 como teste de regressão

Fluxo de um dia normal: o consultor-dev abre PR → pre-commit já barrou segredo local → os gates 1-5 rodam em ~6 min → revisão de segurança (CODEOWNERS) → merge → imagem escaneada, testada por DAST, assinada → alguém do time aprova o deploy → produção só aceita o digest assinado.

1.4. Evidência de execução: antes × depois

Todas as ferramentas foram executadas localmente sobre **a base da Sprint 2** (commit `acead24`) e sobre **o código final**. Relatórios em `docs/evidencias/scans/`.

Ferramenta	Sprint 2 (antes)	Sprint 3 (depois)	O que mudou
pip-audit (SCA)	27 CVEs em 4 pacotes: pyjwt 2.9.0 (7), cryptography 43.0.1 (6), python-multipart 0.0.10 (7), starlette 0.38.6 (7)	0 — "No known vulnerabilities found"	commit <code>b5c674c</code>
Semgrep (227 regras)	1 bloqueante : <code>--forwarded-allow-ips=*</code> no Dockerfile	0	commits <code>31f0618</code> , <code>ee289d2</code>
Bandit	0 médio/alto (2 baixos)	0 médio/alto (22 baixos: <code>random</code> no simulador, <code>try/except</code> de best-effort)	gate é médio+
Gitleaks (histórico)	0 no código da Sprint 2	4 achados na primeira passada, triados : 2 falsos positivos (regex nossa atravessava quebra de linha; JWT <code>alg=none</code> forjado de propósito na demo) e 2 placeholders da Sprint 2 documentados em <code>.gitleaksignore</code> → 0	commit <code>31c1443</code>
Trivy image	primeiro build da Sprint 3: 2 HIGH (msgpack e setuptools vendorizados no pip)	0 HIGH/CRITICAL corrigível (Debian 13.7 + Python)	commit <code>62d2dae</code> : pip removido da imagem final
Trivy config	—	0 misconfig HIGH/CRITICAL em Dockerfile e 3 manifests k8s	<code>4aee75d</code>
Hadolint	2 warnings (DL3045, DL3013)	0	<code>62d2dae</code>
pytest	8 testes, 1 já falhando (máscara de CPF)	75 testes, todos passando (unitários + integração com Postgres); cobertura de linhas 75% no pacote, 86% sem as ferramentas de demo (simulador, gerador de tráfego, alert-sink)	ver <code>pytest.txt</code>

Os achados do "depois" não foram maquiados: o Trivy e o Gitleaks **pegaram problemas reais no próprio trabalho desta sprint**, e a correção virou commit. É o ciclo que o pipeline impõe.

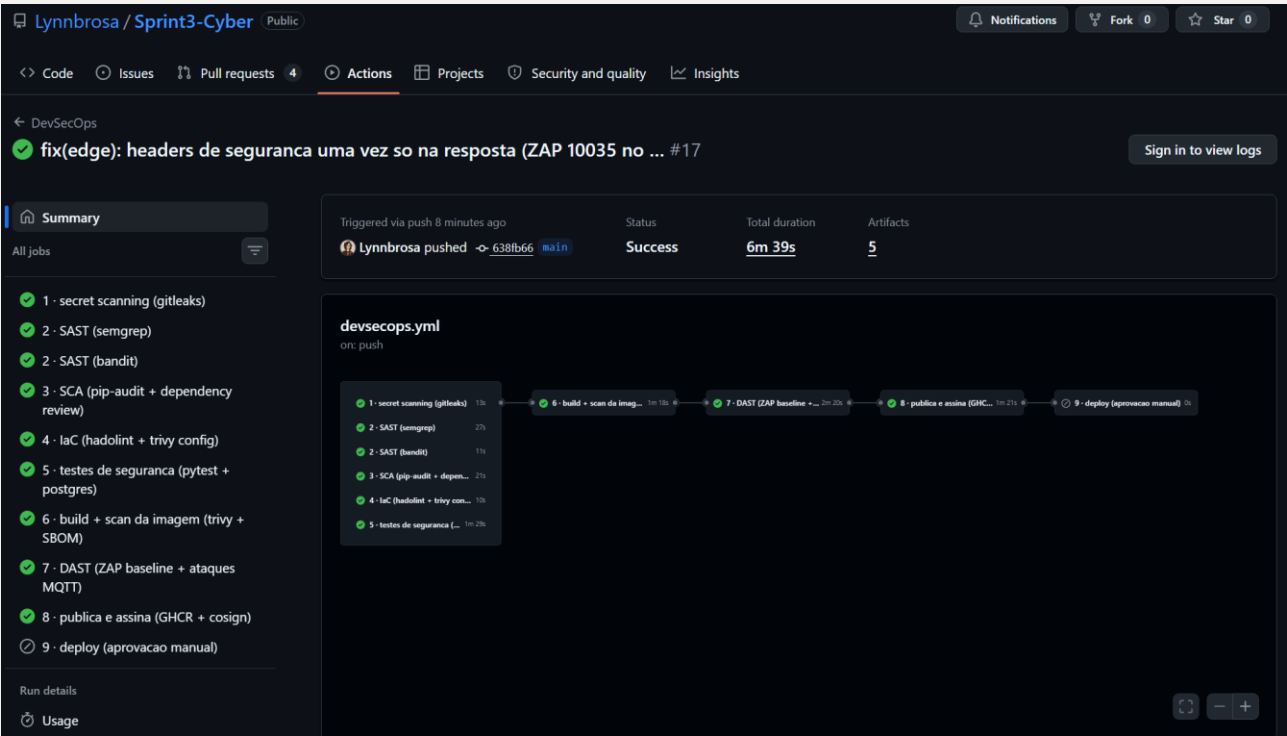
Primeira execução no GitHub (repositório `Lynnbrosa/Sprint3-Cyber`, 26/09/2026 13:08 UTC): gitleaks, laC e os 75 testes contra Postgres passaram no runner. SAST e SCA ficaram vermelhos por **defeito do próprio workflow**, não por achado: o container do semgrep recusa `pip install` (PEP 668) e o bandit não instalava; o pip-audit não cria o arquivo de relatório quando não há vulnerabilidade e o passo que o lia quebrava. A revisão da falha também achou três problemas nos estágios que ainda não tinham rodado (nome da imagem no GHCR precisa ser minúsculo, ZAP quebrava em aviso e não só em FAIL, `compose run` sem `-T`). Tudo corrigido no commit seguinte; o Dependabot já abriu as primeiras verificações de pip, Docker, Compose e Actions.

Com a stack subindo no runner (a tag do Mosquitto fixada aqui só existia na máquina local; a oficial é `2.1.2-alpine`, mesmo digest), **o DAST pegou um achado real**: `Strict-Transport-Security` duplicado (ZAP 10035, FAIL). O nginx e a API mandavam os mesmos cinco cabeçalhos de segurança; com dois HSTS na resposta a

RFC 6797 manda o navegador ler só o primeiro, e uma divergência futura entre os dois valores passaria despercebida. A borda agora esconde os cabeçalhos que vêm da API (`proxy_hide_header`) e passou a mandar também `Cross-Origin-Resource-Policy` e `Cache-Control: no-store`, que o ZAP tinha deixado como aviso.

Na execução seguinte (run #17, commit `638fb66`) **os nove estágios passaram** em 6 min 39 s: a imagem foi publicada em `ghcr.io/lynnbrosa/previopls-security` com assinatura cosign, e o deploy fica parado (skipped) até existir o ambiente `production` com revisores e a variável `DEPLOY_ENABLED`.

O Code scanning do GitHub ainda mostrava 2 alertas `rota-sem-controle-de-acesso` em `auth.py`: são o login e o refresh, públicos de propósito e marcados com `nosemgrep` e o motivo. O gate já os ignorava, mas o semgrep grava a exceção no SARIF (`suppressions: inSource`) e o GitHub abre alerta mesmo assim. O job agora tira do SARIF o que já tem exceção justificada no código; a exceção continua visível no próprio código e passa pela revisão do CODEOWNERS.



Pipeline DevSecOps aprovado no GitHub Actions (run #17)

1.5. O pipeline nas outras frentes da Sprint 3

Os estágios 1 a 3 (segredo, SAST e dependências) rodaram nos repositórios das outras disciplinas, no estado entregue em 27/09/2026. Relatórios em `docs/evidencias/scans/outras-frentes/`.

Repositório	Secret scanning (gitleaks, histórico inteiro)	SAST (semgrep)	SCA
<code>Sprint3-SOA</code> (Java 21, Spring Boot 3.3.4), <code>02e4275</code>	5 alertas: 3 falsos positivos (tokens de demo expirados nas evidências, segredo do perfil de teste) e 2 reais (F1)	<code>p/java</code> , <code>p/owasp-top-ten</code> , <code>p/jwt</code> , <code>p/secrets</code> : 0 em 86 arquivos	85 vulnerabilidades conhecidas (9 críticas, 29 altas) em 92 dependências de runtime, consultadas no OSV; quase todas vêm do Spring Boot 3.3.4 (Tomcat 10.1.30, Spring 6.1.13, Spring Security 6.3.3)

Repositório	Secret scanning (gitleaks, histórico inteiro)	SAST (semgrep)	SCA
<code>Sprint3-mobile</code> (Expo), <code>7c910d6</code>	0	<code>p/typescript</code> , <code>p/react</code> , <code>p/owasp-top-ten</code> , <code>p/secrets</code> : 0 em 64 arquivos	<code>npm audit --omit=dev</code> : 14 moderadas, 0 altas, todas de 2 avisos de origem (<code>uuid</code> só no prebuild; <code>decode-uri-component</code> via <code>expo-router</code> , DoS no parse de URL) sem correção upstream
<code>Sprint3-IA-ML</code> (Python, XGBoost), <code>b8c0ed3</code>	0	<code>p/python</code> , <code>p/owasp-top-ten</code> , <code>p/secrets</code> e <code>bandit</code> : 0; regra do projeto: 1 (F3)	<code>pip-audit</code> : 0, mas o <code>requirements.txt</code> usa <code>>=</code> e o resultado só vale para as versões de hoje

Os pacotes prontos não acharam nada no código das três frentes; os achados vieram do SCA, da triagem do gitleaks e de uma regra escrita para o projeto. Nenhum pacote do registry do semgrep cobre `joblib.load`, então a regra `modelo-carregado-sem-conferir-hash` entrou em `.semgrep/`: ela pega o `inferencia.py` e não acusa uma função que confere o SHA-256 antes do load (testada com um caso positivo e um negativo).

#	Frente	Achado	Risco	Correção proposta
F1	SOA	<code>application.yml</code> tem valor padrão para <code>JWT_SECRET</code> e <code>APP_CRYPTO_KEY</code> (o mesmo do <code>.env.example</code> público) e o app sobe com ele se a variável faltar	quem lê o repositório assina JWT HS256 como admin e decifra a PII	tirar o padrão (<code>{JWT_SECRET}</code> sem valor) e recusar o boot fora do perfil de dev, como a API de segurança já faz (§2.2)
F2	SOA	Spring Boot 3.3.4, fora de suporte	CVEs críticas no Tomcat embutido e no Spring Security (bypass de autorização, request smuggling)	subir o <code>spring-boot-starter-parent</code> para 3.5.16, rodar a suíte do SOA e ligar o Dependabot <code>maven</code> no repositório
F3	IA/ML	<code>joblib.load</code> do modelo sem conferir hash (<code>src/inferencia.py:43</code>)	quem troca o <code>.joblib</code> executa código na máquina que carrega o modelo	gravar o SHA-256 no treino (fora da pasta do modelo) e conferir antes do load
F4	Mobile	APK com <code>usesCleartextTraffic: true</code> ; logout que não apaga o cache de leads do AsyncStorage	MITM em rede aberta; o próximo usuário do aparelho vê os leads do anterior	cleartext desligado no build de produção e <code>pinning</code> ; <code>Local.apagarPrefixo('previopls:cache:')</code> no <code>sair()</code>

As correções ficam com o repositório de cada disciplina e não foram aplicadas a partir deste. A troca do Spring Boot (F2) não foi validada aqui: a suíte do SOA precisa rodar no ambiente do time antes do merge.

1.6. Rotina contínua ligada ao pipeline

Rotina	Onde está	Frequência
Dependabot: pip, Dockerfile, docker-compose, GitHub Actions	<code>.github/dependabot.yml</code>	semanal (segunda 06:00 BRT); patch/minor agrupados
Pipeline agendado (SCA + DAST sem commit)	<code>schedule</code> no workflow	semanal
SBOM de cada imagem publicada	artefato <code>sbom</code> do job 6	a cada build

2. Segurança em código e infraestrutura (peso 2,5)

2.1. Correções reais no código (antes → depois)

A base da Sprint 2 ([acead24](#)) passou por revisão e testes de ataque. Tudo abaixo era **defeito real em produção** e cada item tem commit e teste automatizado.

#	Problema encontrado	Impacto	Correção	Commit	Teste
1	Lockout nunca disparava: a falha de login levantava exceção antes do <code>commit()</code> e o rollback apagava o <code>login_attempt</code> e o <code>LOGIN_FAILED</code>	força bruta ilimitada e sem trilha	persistir antes do <code>raise</code> ; normalizar e-mail; usuário inexistente paga o mesmo bcrypt (sem enumeração por tempo)	d370452	<code>test_lockout_apos_5_falhas...</code> , <code>test_senha_errada_401...</code>
2	Rate limit global (100/min) não era aplicado: faltava o <code>SlowAPIMiddleware</code>	só login e LLM tinham limite	middleware registrado	31f0618	<code>test_limite_global_vale_pra_qualquer_rota</code>
3	X-Forwarded-For forjável: uvicorn com <code>--forwarded-allow-ips=*</code> e nginx anexando o XFF do cliente	trocar o header a cada request = bucket novo no rate limit + IP falso na auditoria	nginx sobrescreve com <code>\$remote_addr</code> ; uvicorn só confia no IP do nginx	31f0618	<code>test_xff_forjado_so_funcionava_com_trust_em_todos</code> + demo (109 de 130 barradas)
4	Replay do webhook HMAC dentro da janela de 5 min; <code>SIGNATURE_REJECTED</code> nunca gravado	cadastro reenviado N vezes	<code>hmac_nonces</code> (PK na assinatura) + auditoria de toda recusa	71c519a	<code>test_replay_da_mesma_requisicao_assinada_e_recusado</code>
5	Máscara de CPF vazava 3 dígitos no log (regex 3+5+3); o teste da Sprint 2 já falhava	PII parcial em log	regex 3+6+2; redação por nome de campo (senha, token...); JWT solto vira <code>***JWT***</code>	ac74137	<code>test_logging_masks_cpf_and_email</code>
6	X-Request-Id aceitava qualquer texto até 64 chars	forjar campos no log/auditoria	charset <code>[A-Za-z0-9._-]{8,64}</code>	ac74137	<code>test_request_id_malicioso_e_trocado</code>
7	Logout não invalidava o access token (valia até o <code>exp</code>)	token roubado sobrevivia ao logout	denylist por <code>jti</code> (<code>revoked_tokens</code>)	d03e3d8	<code>test_logout_invalida_o_access_na_hora</code>
8	Refresh reutilizado não era detectado	sessão roubada convivia com a legítima	reuso derruba a família inteira + <code>REFRESH_REUSE_DETECTED</code> + webhook	d03e3d8	<code>test_reuso_de_refresh_derruba_todas_as_sesoes</code>
9	403 de RBAC não ia pra trilha; 401 sem <code>WWW-Authenticate</code> (o handler global descartava os headers)	tentativa de escalação invisível	<code>FORBIDDEN_ACCESS</code> auditado; handler repassa headers	d370452 , 777aede	<code>test_403_por_papel_vira_forbidden_access...</code>
10	27 CVEs nas dependências	ver §1.4	versões atuais pinadas	b5c674c	<code>pip-audit = 0</code>
11	404/405 e erros do nginx fora do contrato de erro (<code>{"detail"}</code>) / página HTML)	vaza tecnologia, quebra cliente	handler no <code>StarletteHTTPException</code> + <code>error_page</code> JSON no nginx	3c018ca	<code>test_rota_inexistente_e_metodo_errado...</code>
12	Alerta da primeira ocorrência nunca disparava (séries Prometheus nasciam com o valor final)	1º ataque passava sem alerta	todas as séries de segurança nascem com 0	57b122c	<code>test_series_de_seguranca_nascem_zeradas</code> + §3.5

2.2. Criptografia local (dados em repouso)

Dado	Como fica gravado	Onde
CPF	Fernet (AES-128-CBC + HMAC-SHA256, não determinístico) + <code>cpf_hash</code> = HMAC-SHA256 com <i>pepper</i> para busca sem decifrar	<code>app/core/crypto.py</code>
E-mail, telefone	Fernet	idem
Resposta da API	só a versão mascarada (<code>123.***.***-09, m***@example.com, ****8888</code>)	<code>cliente_to_output</code>
Datasets de ML/BI	pseudônimo irreversível <code>pseudo_<sha256></code>	<code>pseudonymize()</code>
Senha	bcrypt custo 12	<code>hash_password</code>
Backup	<code>pg_dump</code> cifrado com AES-256 (PBKDF2, 200 mil iterações) + sha256	<code>scripts/backup.sh</code>
Token no celular	Keychain/Keystore via <code>expo-secure-store</code> (app do consultor)	monorepo

Rotação de chave sem downtime (novo nesta sprint, commit `dad3069`):

```
# app/core/crypto.py – a primeira chave de FERNET_KEYS cifra, as outras só decifram
self._fernet = MultiFernet([Fernet(k.encode()) for k in settings.fernet_key_list])
```

```
FERNET_KEYS="<nova>,<antiga>" python -m app.cli.rotate_keys # recifra em lotes, grava KEY_ROTATED
FERNET_KEYS="<nova>" # a antiga sai do cofre
```

Teste: `test_rotacao_recifra_e_chave_antiga_pode_sair` prova que depois da rotação o dado abre com a chave nova e **não abre mais** com a antiga.

Boot recusa segredo fraco em produção (`Settings._segredos`): com `APP_ENV=production` a API não sobe se o pepper ou o segredo HMAC tiverem menos de 32 caracteres, parecerem de exemplo (`troque...`, `example`, `test...`), forem iguais entre si, ou se o CORS não tiver whitelist. O `.env.example` não tem nenhum valor: `scripts/bootstrap_env.py` gera tudo aleatório por clone.

2.3. Hardening da API

Rate limit em duas camadas (IP real, depois da correção 3):

Camada	Global	Login	LLM	Resposta
nginx <code>limit_req</code>	100/min (burst 20)	5/min (burst 2)	10/min	429 JSON + <code>Retry-After</code>
slowapi na API	100/min	5/min	10/min	429 JSON
lockout por conta	—	5 falhas em 60 s → 15 min	—	401 "Conta temporariamente bloqueada"

Resultado da demo (`api-ataques.txt`): força bruta em rajada → `[401, 429, 429, ...]`; força bruta lenta abaixo do limite → **lockout na 6ª tentativa**; 130 requisições com `X-Forwarded-For` diferente → **109 barradas**.

Validação de entrada: todo DTO Pydantic com `extra="forbid"` (mass assignment), limites de tamanho, regex de CPF/VIN/placa, enums nos filtros (SQLi no filtro vira 422), corpo de 1 MB no nginx. Demo: `SQLi no filtro: 422`; `campo extra no cadastro: 422`.

JWT seguro:

Controle	Implementação
Algoritmo	RS256 com chave de 3072 bits; <code>algorithms=["RS256"]</code> fixo (sem <code>alg=none</code> /confusão)
Rotação de chave	header <code>kid</code> ; a chave anterior fica aceita só durante a janela (<code>JWT_PREVIOUS_PUBLIC_KEY_PATH</code>); <code>kid</code> desconhecido é recusado antes da verificação
Claims obrigatórias	<code>exp</code> , <code>iat</code> , <code>iss</code> , <code>aud</code> , <code>sub</code> , <code>jti</code> , <code>type</code> ; <code>type</code> impede usar refresh como access
Expiração	access 15 min; refresh 7 dias com rotação a cada uso
Revogação	denylist por <code>jti</code> no logout; <code>sessoes_revogadas_em</code> invalida tudo emitido antes (kill switch)
Roubo de refresh	reuso detectado → família revogada, alerta crítico
Transporte	só em header <code>Authorization</code> (sem cookie na API → sem CSRF)

Demo: 24 tokens forjados/errados → todos 401; reuso do refresh antigo → 401 **e o refresh legítimo também morre** (401).

HMAC nos webhooks de faturamento: `X-Signature = HMAC-SHA256(segredo, timestamp.corpo)`, janela de 5 min, `compare_digest`, uma assinatura vale uma vez. Demo: original 201; replay 401; corpo adulterado 401; timestamp velho 401; sem assinatura 401.

Headers e erros: HSTS preload, CSP `default-src 'none'` na API, `X-Frame-Options DENY`, `nosniff`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Resource-Policy` e `Cache-Control: no-store` (resposta com PII), cada um uma vez só na resposta, sem `Server` com versão; Swagger desligado em produção; erro sempre `{"error": {"code", "message"}}` com `incident=<id>` e nunca stack trace.

2.4. Controle de acesso por perfil

Implementado com `Depends(requires_role(...))` em cada rota; a regra de SAST `rota-sem-controle-de-acesso` quebra o build se alguém criar rota sem ela (rota pública precisa de justificativa explícita, como login e refresh).

Recurso	Consultor	Analista (gestor)	Admin
GET <code>/v1/leads</code> , GET <code>/v1/leads/{id}</code>	● OK	● OK	● OK
PATCH <code>/v1/leads/{id}</code>	● OK	✗ 403	● OK
POST <code>/v1/clientes</code> (+ HMAC)	✗ 403	✗ 403	● OK
GET <code>/v1/admin/audit-log</code>	✗ 403	● OK	● OK
POST <code>/v1/admin/usuarios/{id}/revogar-sessoes</code>	✗ 403	✗ 403	● OK
POST <code>/v1/llm-assist</code>	✗ 403	● OK	● OK

A matriz inteira é testada (`test_matriz_rbac`, 10 casos) e todo 403 grava `FORBIDDEN_ACCESS`. Na demo o consultor tentou 9 rotas de admin → 9 × 403 → alerta `TentativasDeEscalacaoDePrivilegio`. A rotina mensal `auditoria_permissoes` apontou exatamente esse consultor (\$4.6).

2.5. Segurança MQTT/TLS para IoT

Veículos conectados publicam telemetria em `ford/telemetria/<VIN>`.

```
# iot/mosquitto/mosquitto.conf (trecho)
listener 8883
require_certificate true          # mTLS: sem certificado da CA interna não passa do handshake
use_identity_as_username true    # o CN do certificado (= VIN) vira o usuário
tls_version tlsv1.2             # piso TLS 1.2 (negociou TLS 1.3 na prática)
allow_anonymous false
max_packet_size 4096
```

```
# iot/mosquitto/acl – o carro só publica no próprio tópico; o ingestor só lê
user ingestor
topic read ford/telemetria/+
pattern write ford/telemetria/%u
```

Camada	Controle
Identidade	certificado X.509 por VIN (ECDSA P-256), validade de 90 dias , emitido pela CA interna (<code>scripts/gen_certs.py</code>)
Transporte	só porta 8883 TLS; não existe 1883 em texto claro
Autorização	ACL por CN: um carro comprometido não fala em nome de outro VIN
Conteúdo (ingestor)	schema fechado, faixas (km ≤ 2 mi, velocidade ≤ 300), código OBD-II por regex, VIN do corpo = VIN do tópico, relógio do device (±2 min / 24 h), odômetro regressivo (fraude de km)
LGPD	posição gravada com 2 casas (~1 km) e retenção de 90 dias

Execução real (`iot-ataques.txt`):

```
[normal] 9BFZZ8F7NB000001 leitura 1/6 km=31551: PUBACK Success
[sem-certificado] recusado pelo broker antes do CONNACK (mosquitto: peer did not return a certificate)
[ca-falsa] recusado pelo broker antes do CONNACK (mosquitto: certificate verify failed)
[texto-claro] porta 1883 fechada: ConnectionRefusedError
[tópico-alheio] 9BFZZ8F7NB000001 tentou publicar em ford/telemetria/9BFZZ8F7NB000002: PUBACK Not authorized
[payload-invalid] campo_extra / odometro_negativo / dtc_injecao / relógio_futuro / payload_grande → recusados no ingestor
[odometro-regressivo] 47371 -> 39371 → recusado (motivo odometro-regressivo)
```

Achado durante o teste: no TLS 1.3 o cliente termina o handshake **antes** de o broker validar o certificado dele, então "`connect()` não deu erro" não prova nada. O simulador passou a esperar o CONNACK, e o log do broker confirma que nenhuma das conexões chegou a `New client connected`.

2.6. IaC Security (Dockerfile, Compose, Kubernetes)

Dockerfile (`Dockerfile`):

Prática	Como
Base reproduzível	<code>python:3.12-slim@sha256:f77ac9...</code> (digest; Dependabot abre PR quando muda)
Multi-stage	dependências compiladas no estágio <code>build</code> , só o <code>venv</code> vai pro <code>runtime</code>
Patches do SO	<code>apt-get -y upgrade</code> no build (Trivy recusa CVE com correção)
Superfície mínima	sem pip, sem curl, sem shell de login ; healthcheck em Python
Não-root	<code>USER 10001:10001</code> ; código pertence ao root (processo não reescreve a app)

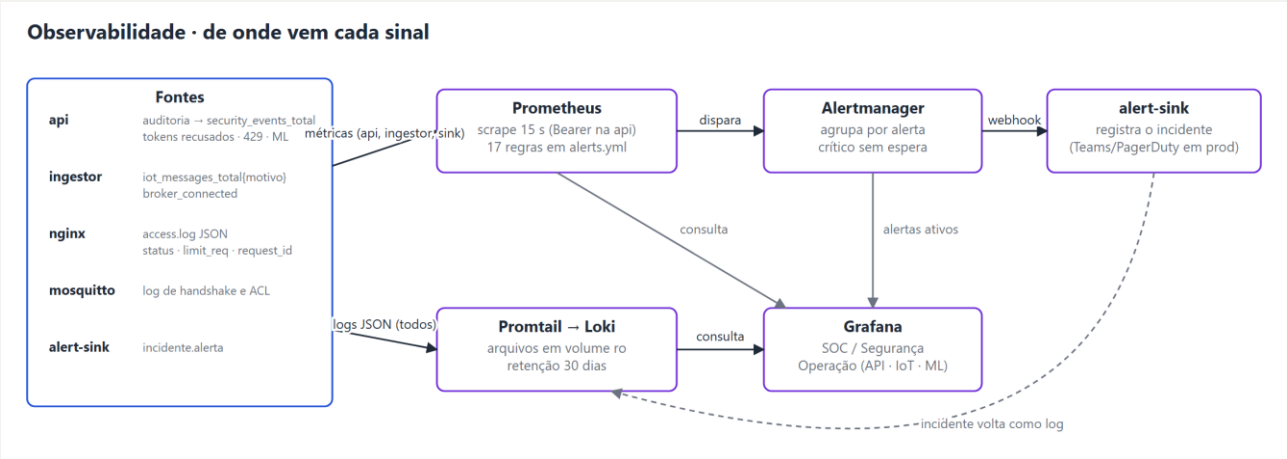
docker-compose (`docker-compose.yml`): todo serviço com `read_only: true`, `cap_drop: [ALL]`, `no-new-privileges`, `mem_limit` e `pids_limit`; banco rodando como uid 70 numa rede `internal: true` (sem rota pra internet); certificado montado **só no container que usa** (o volume da CA não é montado em nenhum serviço de runtime); nginx `nginx-unprivileged` (uid 101); Grafana/Prometheus/Alertmanager só em `127.0.0.1`; promtail sem `docker.sock` (lê arquivo em volume read-only, com a capability mínima `DAC_READ_SEARCH`); `/metrics` com Bearer via `secret` de arquivo.

Kubernetes (`infra/k8s/`): namespace com Pod Security Admission `restricted`; `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities: drop [ALL]`, `seccompProfile: RuntimeDefault`; `automountServiceAccountToken: false`; requests/limits; **NetworkPolicy default-deny** com liberação explícita (`ingress-nginx` e `monitoring` → API; API → Postgres, Redis e DNS); imagem **por digest**; segredos pelo External Secrets (nada de segredo em YAML); `PodDisruptionBudget`; Ingress com TLS do cert-manager.

Resultado: **Trivy config 0 HIGH/CRITICAL**, **Hadolint 0 warnings** (§1.4).

3. Observabilidade, monitoramento e resposta (peso 2,0)

3.1. Arquitetura de observabilidade



Observabilidade

Sinal	Fonte	Coleta	Retenção
Métricas	/metrics da API (Bearer), ingestor :9101, alert-sink :9102	Prometheus (scrape 15 s)	15 dias
Logs JSON	API, ingestor, alert-sink, nginx, Mosquitto	Promtail → Loki (volumes read-only)	30 dias
Trilha de auditoria (LGPD)	tabela audit_logs	Postgres + GET /v1/admin/audit-log	5 anos
Alertas	17 regras em alerts.yml	Alertmanager → alert-sink (Teams/PagerDuty em produção)	—

Todo evento de auditoria também vira métrica (previoplsecurity_events_total{action}), então alerta, dashboard e trilha LGPD saem da mesma fonte, sem código duplicado.

3.2. Logs estruturados

JSON do structlog com request_id correlacionando nginx → API → auditoria, e PII mascarada antes de qualquer escrita. Exemplos reais (logs-exemplo.txt):

```
{
  "action": "LOGIN_FAILED",
  "remote_ip": "127.0.0.1",
  "event": "audit",
  "request_id": "478d4d597a1190d0c8dcd6dd84e73db2",
  "method": "POST",
  "path": "/v1/auth/login",
  "level": "info",
  "timestamp": "2026-09-26T12:12:52.085884Z"
}
{"email": "a***@ford.com", "count_1m": 5, "event": "alert.login_brute_force", "request_id": "ab7dd96f86a68cc5260d9dfc4175a61e", "path": "/v1/auth/login", "level": "warning", "timestamp": "2026-09-26T12:13:43.014609Z"}
{"action": "FORBIDDEN_ACCESS", "actor_role": "consultor", "event": "audit", "path": "/v1/admin/audit-log", "request_id": "6ea45ce8754c18be640243494e589c50", "level": "info"}
{"action": "REFRESH_REUSE_DETECTED", "entity_id": "283b7769-641a-42b3-8182-97168bb0a920", "event": "audit", "path": "/v1/auth/refresh", "level": "info"}
{"actor": "283b7769-641a-42b3-8182-97168bb0a920", "count_1m": 51, "event": "alert.mass_query", "path": "/v1/leads", "level": "warning", "timestamp": "2026-09-26T12:30:47.756012Z"}
{"motivo": "odometro_regressivo", "detalhe": "47371 -> 39371", "vin": "9BFZZZ8F7NB000001", "security": true, "event": "iot.telemetria_rejeitada", "level": "warning"}
{"time": "2026-09-26T12:09:00+00:00", "remote_addr": "172.30.10.1", "method": "GET", "uri": "/metrics", "status": 404, "request_id": "f6d84d1258c8f31aca5a6899fa823c4d", "ssl_protocol": "TLSv1.3"}
{"incidente_id": "d8eaec396465", "origem": "alertmanager", "status": "firing", "alerta": "ForcaBrutaLogin", "severidade": "high", "runbook": "docs/runbooks/RB-01-forca-bruta.md", "inicio": "2026-09-26T12:15:28.41Z"}
```

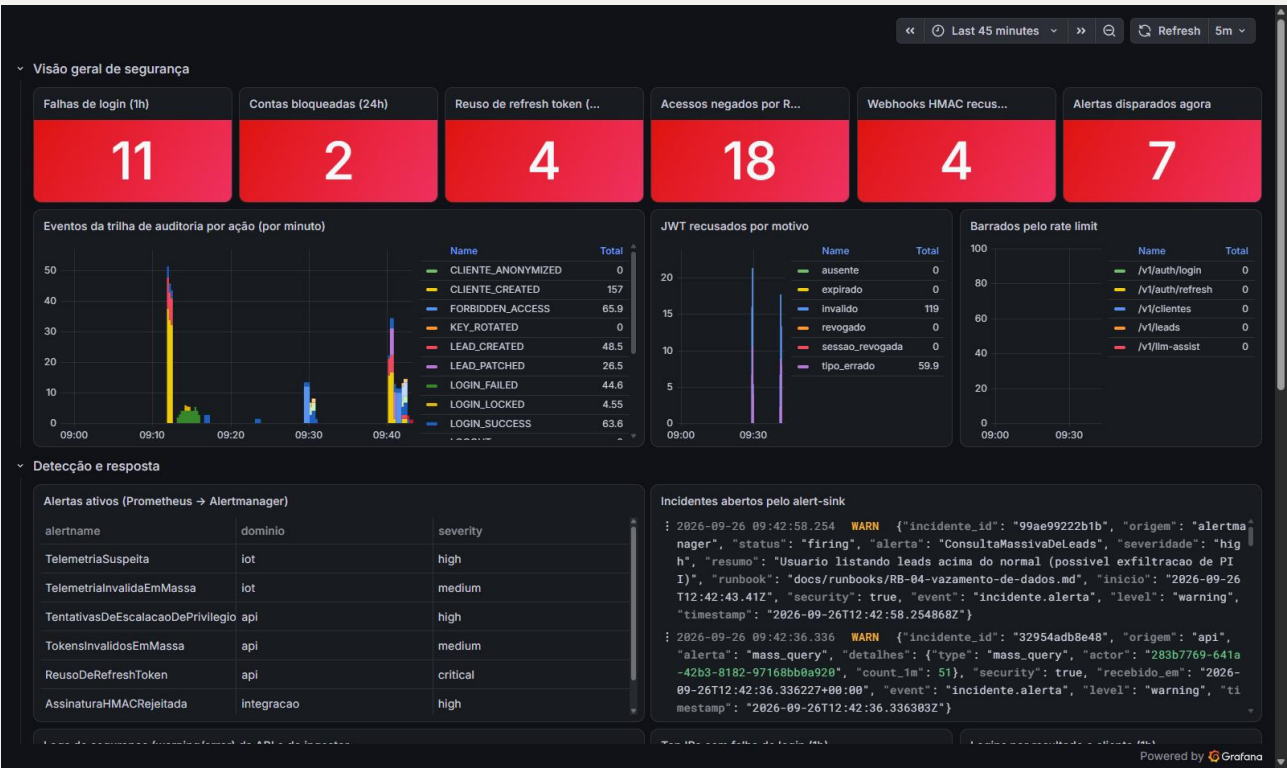
Evento de segurança logado	Onde
login com sucesso/falha/bloqueio, logout, refresh	LOGIN_*, LOGOUT, TOKEN_REFRESHED
alterações críticas	CLIENTE_CREATED, LEAD_PATCHED, PROFILE_CHANGED, SESSIONS_REVOKED, KEY_ROTATED, CLIENTE_ANONYMIZED
ataques e anomalias	UNAUTHORIZED_ACCESS, FORBIDDEN_ACCESS, SIGNATURE_REJECTED, REFRESH_REUSE_DETECTED, MASS_QUERY_DETECTED, rate_limit.exceeded
IoT	iot.telemetria_rejeitada{motivo}, recusas de handshake no log do broker

3.3. Métricas e alertas por domínio

Domínio	Métricas	Alertas (severidade)
API	requisições/latência/status por rota (template, sem cardinalidade explosiva), 429 por rota, JWT recusado por motivo	ForcaBrutaLogin (high), ContaBloqueadaPorLockout (medium), ReusoDeRefreshToken (critical), TentativasDeEscalacaoDePrivilegio (high), TokensInvalidosEmMassa (medium), FloodBarradoPeloRateLimit (medium), TaxaDeErro5xxAlta, LatenciaP95Alta, ComponenteForaDoAr (critical)
Integração	recusas de HMAC	AssinaturaHMACRejeitada (high)
Dados	consulta massiva	ConsultaMassivaDeLeads (high)
Mobile	previopls_logins_total{result, client} a partir do header X-Client: mobile/web/integracao; erros de app em Sentry/Crashlytics (plano)	falhas de login por tipo de cliente no painel SOC
IoT	mensagens aceitas/rejeitadas por motivo, broker conectado, marcos de revisão	TelemetriaSuspeita (high), TelemetriaInvalidaEmMassa (medium), IngestorSemBroker (high)
ML	classificações por perfil, latência de inferência, histograma de score	InferenciaLenta (medium), DriftNoPerfilPrevisto (medium)

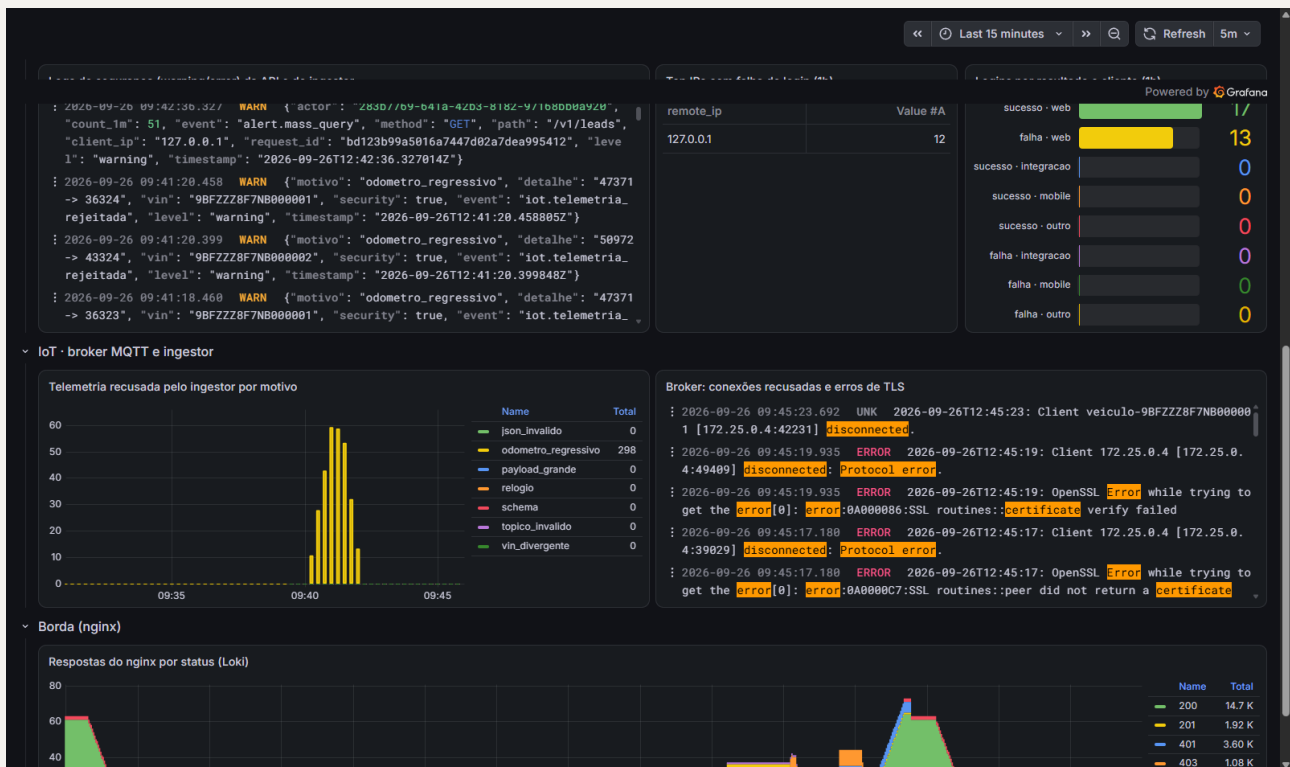
3.4. Dashboards (prints da execução)

PrevioPLS · SOC / Segurança — topo: contadores, eventos da trilha, JWT recusados, alertas ativos e incidentes registrados:



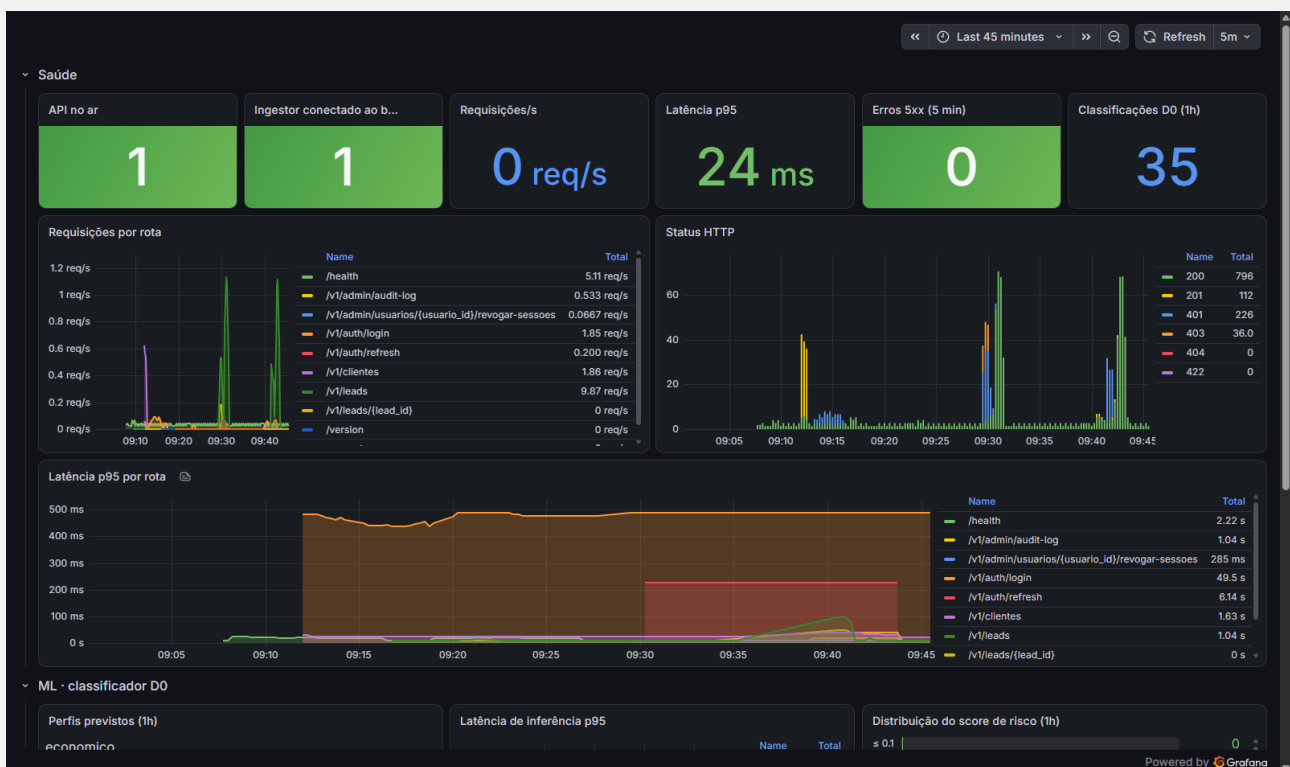
Dashboard SOC: contadores, eventos da trilha, JWT recusados, alertas ativos e incidentes

SOC (parte de baixo) — logs de segurança, top IPs, logins por cliente, telemetria recusada e recusas de TLS no broker:

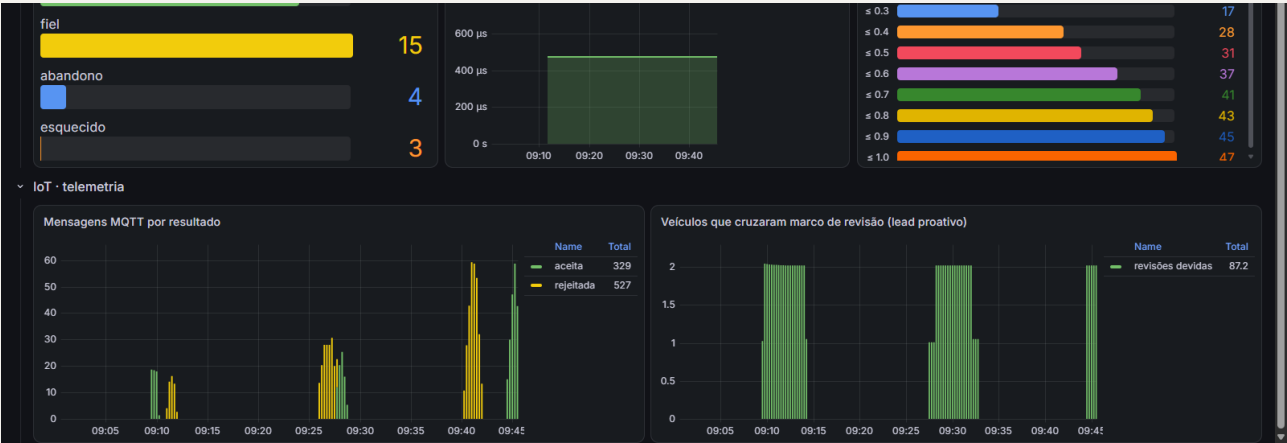


Dashboard SOC, parte de baixo: logs de segurança, telemetria recusada e recusas de TLS no broker

PrevioPLS · Operação — saúde, requisições, status, latência, classificador D0 e telemetria:

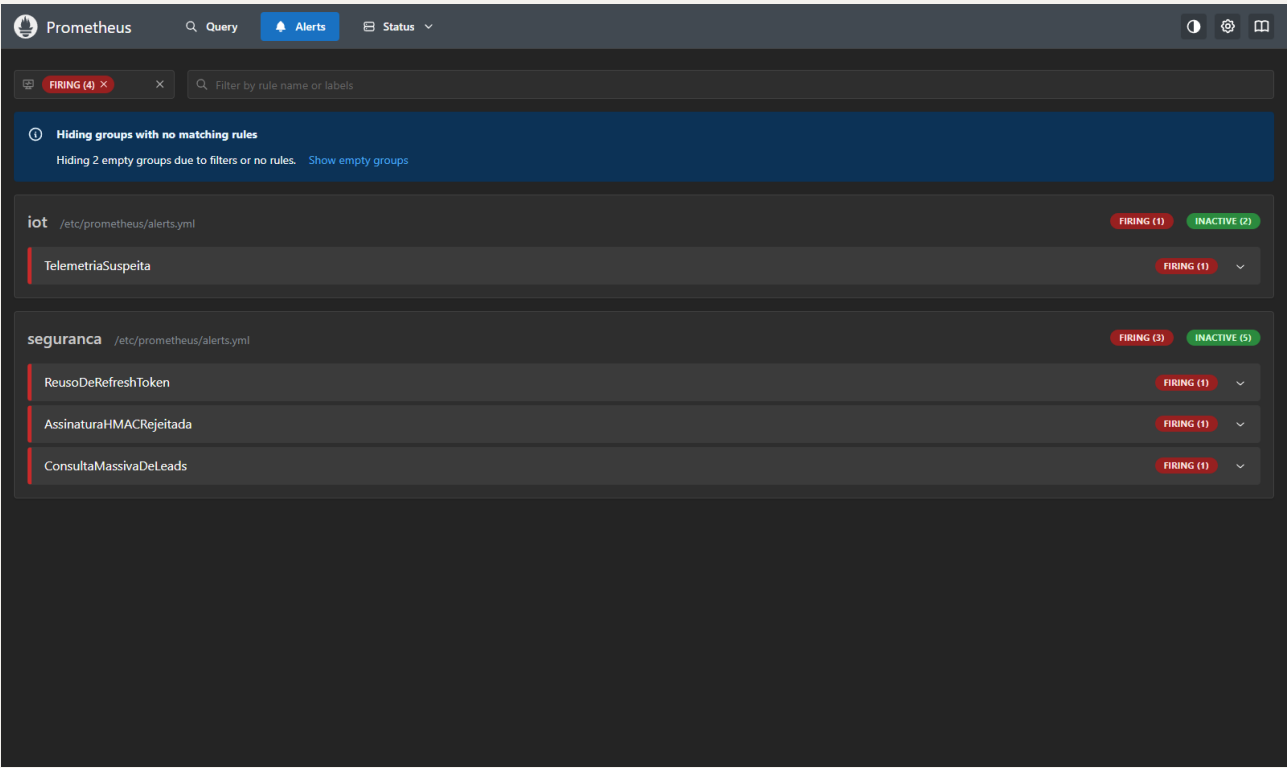


Dashboard de Operação: saúde, requisições, status HTTP e latência



Dashboard de Operação: classificador D0 (perfis, latência, score) e telemetria MQTT

Prometheus · alertas disparando depois da segunda rodada de ataques:



Prometheus: alertas disparando depois da segunda rodada de ataques

3.5. Plano de resposta a incidentes



Resposta a incidentes

Fase	O que fazemos	Ferramenta
Detecção	alerta do Prometheus, webhook da própria API (brute force, reuso de refresh, consulta massiva), dashboard SOC	Alertmanager → alert-sink → Loki
Análise	abrir o runbook indicado no alerta, reconstruir a linha do tempo pelo request_id, medir escopo	audit_logs, LogQL, runbooks
Contenção	derrubar sessões, bloquear IP, revogar certificado do VIN, trocar segredo HMAC	POST /v1/admin/usuarios/{id}/revogar-sessoes, deny no nginx, CRL no broker
Erradicação	corrigir a causa pelo pipeline (nada de hotfix manual em produção), rotacionar chave de PII	rotate_keys, PR + gates
Recuperação	rollback para imagem assinada anterior, restore do backup, monitorar a conta/VIN	cosign verify, restore_test.sh
Pós-incidente	relatório, causa raiz, ajuste de alerta e teste de regressão; comunicação LGPD (§4.4)	ticket + PR

Sete runbooks cobrem os 17 alertas (docs/runbooks/): força bruta, sessão comprometida, integração adulterada, vazamento de dados pessoais, dispositivo IoT, indisponibilidade e modelo de ML. Severidade define prazo: crítico reconhecido em 15 min e contido em 1 h; alto em 30 min / 4 h; médio em 4 h / 1 dia útil.

Exercício executado (26/09/2026, stack local) — linha do tempo real, em UTC:

Hora	Fato	Fase
12:13:43	5ª senha errada no admin → LOGIN_LOCKED; a API manda login_brute_force pro alert-sink	detecção (automática) + contenção (lockout)
12:14:13 / 12:15:28	ContaBloqueadaPorLockout e ForcaBrutaLogin disparam e chegam ao alert-sink	detecção
12:29:28	consultor testa 9 rotas de admin → TentativasDeEscalacaoDePrivilegio	detecção
12:29:58	refresh antigo reutilizado → família revogada automaticamente, webhook refresh_reuse	contenção automática
12:30:13	ReusoDeRefreshToken (critical) dispara; roteamento crítico sem espera	detecção
12:30:47	51ª listagem de leads em 1 min → MASS_QUERY_DETECTED	detecção
12:33–12:37	análise: AssinaturaHMACRejeitada e ConsultaMassivaDeLeads não dispararam, apesar de os eventos estarem na trilha. Causa: série do Prometheus nascia com o valor final e o increase() não conta a primeira amostra	pós-incidente
12:39	correção no ar e teste de regressão (commitada às 12:47 como 57b122c); séries nascem zeradas	erradicação
12:41:58 / 12:42:43	segunda rodada: AssinaturaHMACRejeitada e ConsultaMassivaDeLeads disparam	recuperação / validação

O exercício encontrou uma falha real de detecção, e ela saiu corrigida com teste. É exatamente o ciclo que o plano propõe.

4. Compliance, riscos e segurança contínua (peso 2,5)

4.1. Revisão final dos riscos — STRIDE por componente

Legenda: ● **OK** mitigado e testado · ● **Parcial** mitigado parcialmente / depende de produção · ● **Residual** risco residual aceito.

Componente	Ameaça (STRIDE)	Controle	Status
API	S força bruta / credential stuffing	rate limit 2 camadas + lockout + bcrypt 12 + alerta	● OK
API	S JWT forjado (alg=none, confusão de chave)	RS256 fixo, kid, claims obrigatórias	● OK
API	S sessão roubada	TTL 15 min, rotação, detecção de reuso, denylist, kill switch	● OK
API	T mass assignment / injeção	extra="forbid", enums, ORM parametrizado	● OK
API	R usuário nega ação	audit_logs com ator, IP real, request_id, UA	● OK
API	I PII em resposta/log	mascaramento no DTO e no log, Fernet em repouso	● OK
API	D flood	limites por IP real, corpo 1 MB, timeouts, paginação ≤ 100	● OK (● Parcial Redis para multi-réplica)
API	E consultor vira admin	RBAC por rota + regra SAST + matriz testada	● OK
API	E/I consultor lê lead de outra concessionária (BOLA)	hoje o escopo é por papel, não por concessionária	● Residual roadmap
Integração	T/S webhook adulterado ou replay	HMAC + janela + nonce	● OK
Mobile	I token no aparelho	SecureStore (Keychain/Keystore)	● OK
Mobile	I cache de leads em AsyncStorage (texto claro), que hoje sobrevive ao logout (F4)	limpar no logout + cifrar cache	● Parcial
Mobile	T MITM: o APK da sprint aceita HTTP (F4)	cleartext desligado no build de produção; pinning	● Parcial
IoT	S device falso / clonado	mTLS com CA interna, cert de 90 dias, CRL	● OK
IoT	T telemetria adulterada (km)	validação + odômetro regressivo + alerta	● OK
IoT	E device publica em nome de outro	ACL por CN	● OK
IoT	I localização do titular	~1 km + 90 dias	● OK
Dados	I dump do banco	PII cifrada, chave fora do banco, rede interna	● OK (● Parcial chave em KMS)
Dados	T operador adultera audit_logs	app é o único escritor; papéis separados no Postgres	● Parcial
ML	T envenenamento de entrada / troca do modelo	HMAC na entrada, alerta de drift, hash do artefato	● Parcial
Observabilidade	I log vazando PII	processor de máscara antes do encoder	● OK

Riscos do próprio DevSecOps (supply chain):

Ameaça	Controle
dependência maliciosa ou vulnerável	pip-audit estrito, dependency review, versões pinadas, Dependabot com revisão humana
action do GitHub comprometida	pin por SHA, permissões mínimas por job
segredo exposto no CI	segredos só no environment de deploy; PR de fork sem segredo; gitleaks
imagem adulterada no registry	publicação por digest, cosign keyless, <code>cosign verify</code> antes do deploy
alteração maliciosa do pipeline	CODEOWNERS em <code>.github/</code> , branch protegida, revisão obrigatória
base de imagem vulnerável	digest pinado + Trivy + Dependabot <code>docker</code>

4.2. Mapeamento OWASP ASVS 4.0.3

Capítulo	Requisitos	Como atendemos	Status
V1 Arquitetura	1.1.2 threat model; 1.4.1 controle de acesso no servidor	STRIDE (§4.1), zonas de confiança, RBAC só no backend	● OK
V2 Autenticação	2.2.1 anti-automação; 2.4.1 hash de senha; 2.1.1 senha ≥ 12	rate limit + lockout; bcrypt 12; 12+ caracteres para usuário de produção (6 só no seed de demo)	● OK / ● Parcial MFA
V3 Sessão	3.3.1 logout invalida; 3.3.3 encerrar todas as sessões; 3.5.3 token assinado	denylist por <code>jwt</code> ; kill switch; RS256	● OK
V4 Acesso	4.1.1 no servidor; 4.1.3 menor privilégio; 4.1.5 falha segura; 4.2.1 IDOR	<code>requires_role</code> ; analista só leitura; 403 padrão; UUID não sequencial	● OK / ● Residual escopo por concessionária
V5 Validação	5.1.3 allowlist; 5.1.4 tipagem; 5.3.4 consulta parametrizada	Pydantic <code>extra="forbid"</code> , regex, enums; SQLAlchemy	● OK
V6 Criptografia	6.2.1 falha segura; 6.2.2 algoritmo aprovado; 6.4.1 gestão de chave; 6.4.2 rotação	<code>InvalidToken</code> → erro; Fernet/AES/HMAC-SHA256/RS256; MultiFernet + <code>kid</code>	● OK / ● Parcial KMS
V7 Logs e erros	7.1.1 sem credencial em log; 7.1.3 evento de segurança; 7.1.4 contexto; 7.4.1 erro genérico	máscara + redação por campo; trilha completa; <code>request_id</code> ; <code>incident=</code>	● OK
V8 Proteção de dados	8.3.4 inventário de dado sensível; 8.3.8 retenção; 8.2.2 dado no cliente	§4.4; anonimização 5 anos, telemetria 90 dias; SecureStore	● OK / ● Parcial cache mobile
V9 Comunicação	9.1.1 TLS em tudo; 9.1.2 cifras fortes; 9.1.3 só TLS 1.2+; 9.2.1 certificado confiável	nginx e MQTT; Mozilla <i>intermediate</i> ; TLS 1.1 recusado (testado); mTLS	● OK / ● Parcial mTLS interno
V10 Código malicioso	10.3.2 integridade de artefato	cosign + digest; SAST	● OK
V11 Lógica de negócio	11.1.4 anti-automação de fluxo sensível	consulta massiva, LLM 10/min	● OK
V12 Arquivos	upload	sem upload; corpo ≤ 1 MB	n/a
V13 API	13.1.3 URL sem dado sensível; 13.2.1 métodos restritos; 13.2.6 content-type	CPF nunca em URL; <code>limit_except</code> ; JSON only	● OK
V14 Configuração	14.2.1 componente atualizado; 14.2.2 sem componente desnecessário; 14.3.2 debug off; 14.4 headers; 14.5.3 CORS	SCA; imagem sem pip/curl; Swagger off em prod; headers OWASP; whitelist exigida em prod	● OK

4.3. OWASP API Security Top 10 (2023) e Mobile Top 10 (2024)

API Top 10:

Risco	Situação no PrevioPLS	Status
API1 BOLA	IDs UUID, acesso por papel; falta restringir o lead à concessionária do consultor	● Residual roadmap
API2 Broken Authentication	RS256, logout, rotação, reuso detectado, denylist	● OK
API3 Broken Object Property Level Auth	<code>extra="forbid"</code> na entrada; DTO de saída só com PII mascarada	● OK
API4 Unrestricted Resource Consumption	rate limit 2 camadas, 1 MB, paginação ≤ 100, limites do MQTT	● OK
API5 Broken Function Level Auth	<code>requires_role</code> em toda rota + regra SAST + matriz testada	● OK
API6 Unrestricted Access to Sensitive Business Flows	detecção de consulta massiva, LLM limitado	● OK
API7 SSRF	nenhuma URL vem do usuário; webhook de alerta só por configuração	● OK
API8 Security Misconfiguration	headers, docs off em prod, erro padronizado, IaC escaneado	● OK
API9 Improper Inventory Management	<code>/v1</code> versionado, OpenAPI, SBOM por build	● OK
API10 Unsafe Consumption of APIs	core/ml-api com timeout e rede interna; resposta do LLM tratada como não confiável; HMAC na integração	● OK

Mobile Top 10 (app do consultor, Expo/React Native, `Sprint3-mobile`):

Risco	Situação	Ação
M1 Improper Credential Usage	JWT e papel no <code>expo-secure-store</code> ; nenhuma chave no bundle (só <code>EXPO_PUBLIC_API_URL</code>)	● OK
M2 Inadequate Supply Chain Security	<code>npm audit</code> : 14 moderadas de 2 avisos sem correção upstream, 0 altas (\$1.5)	● Parcial monitorado; <code>npm audit</code> /OSV no pipeline do app
M3 Insecure Authentication/Authorization	autorização só no servidor; 401 limpa a sessão local	● OK; desbloqueio por biometria (backlog)
M4 Insufficient Input/Output Validation	servidor valida tudo; RN escapa texto	● OK
M5 Insecure Communication	APK com <code>usesCleartextTraffic: true</code> e URL padrão <code>http://10.0.2.2:5000</code> para a API local da demo; a tela de servidor avisa quando está sem TLS	● Parcial build de produção falha se a URL não for <code>https</code> ; <code>usesCleartextTraffic=false</code> ; <code>certificate pinning</code>
M6 Inadequate Privacy Controls	API só entrega PII mascarada	● OK
M7 Insufficient Binary Protections	Hermes + R8/ProGuard no release; sem segredo no binário	● OK
M8 Security Misconfiguration	<code>android:allowBackup</code> e componentes exportados	● Parcial <code>allowBackup=false</code> , MobSF no APK
M9 Insecure Data Storage	token no SecureStore (<code>WHEN_UNLOCKED_THIS_DEVICE_ONLY</code>); cache de leads no AsyncStorage, que o logout não apaga	● Parcial cifrar o cache com chave no SecureStore e limpar no logout
M10 Insufficient Cryptography	nada de cripto própria; Keystore/Keychain	● OK

4.4. LGPD — dados pessoais, telemetria e localização

Papéis: **a Ford é a controladora** e o PrevioPLS atua como **operador** (art. 5º, VI e VII); a relação é regida por contrato de operação com as medidas técnicas deste documento (art. 39).

Dado	Finalidade	Base legal (art. 7º)	Proteção	Retenção
Nome	contato do consultor, lead	execução de contrato / legítimo interesse (pós-venda)	acesso por papel	5 anos → anonimização
CPF	identificar e deduplicar o cliente	execução de contrato	Fernet + hash com pepper; mascarado na resposta	5 anos → anonimização
E-mail, telefone	contato para revisão	legítimo interesse; marketing só com consentimento	Fernet; mascarado	5 anos
VIN, placa, modelo	vincular veículo ao cliente	execução de contrato	acesso por papel	5 anos
Telemetria (km, falhas OBD)	manutenção preditiva / lead de revisão	consentimento no contrato do serviço conectado	mTLS, ACL, validação	90 dias
Localização	regionalizar a oferta	consentimento	generalizada (~1 km)	90 dias
Perfil / score do ML	priorizar leads	legítimo interesse, com revisão humana	pseudônimo em dataset; consultor decide (art. 20)	5 anos
IP, user-agent, trilha	segurança e prova (art. 37 e 46)	legítimo interesse / obrigação	máscara em log; trilha só admin/analista	log 30 dias; trilha 5 anos

Artigo	Atendimento
Art. 6º (necessidade, segurança, prevenção)	coleta mínima no D0, localização generalizada, controles das §§2 e 3
Art. 18 (direitos do titular)	pedidos chegam pela Ford (controladora); a API atende acesso e correção pelo core e eliminação pela anonimização irreversível
Art. 20 (decisão automatizada)	o score só prioriza a fila; o contato e a oferta são decisão do consultor; o perfil pode ser revisto
Art. 37 (registro das operações)	<code>audit_logs</code> com quem, o quê, quando, de onde
Art. 46 (medidas de segurança)	criptografia, RBAC, TLS/mTLS, monitoramento, backup
Art. 48 (incidente)	RB-04: operador avisa a Ford no mesmo dia; a Ford comunica ANPD e titulares em até 3 dias úteis quando há risco relevante (Resolução CD/ANPD nº 15/2024)

4.5. Plano de segurança contínua

Rotina	Frequência	Como	Responsável	Evidência
Revisão de dependências	semanal	Dependabot (pip, Docker, Compose, Actions) + pipeline agendado com pip-audit; PR revisado por humano	dev de plantão	<code>dependabot.yml</code>
Testes de segurança	a cada push/PR + semanal	pipeline completo (SAST, SCA, IaC, imagem, DAST, 75 testes)	pipeline	§1.4
Pentest / exercício	trimestral	<code>trafego</code> + <code>simulador</code> (ataques de API e MQTT) e revisão do threat model	líder de segurança	<code>api-ataques.txt</code> , <code>iot-ataques.txt</code>
Auditoria de permissões	mensal	<code>python -m app.cli.auditoria_permissoes</code> (sai com código 1 quando há pendência)	líder de segurança	abaixo
Backup	diário	<code>scripts/backup.sh</code> (AES-256, sha256, retenção 7 dias; produção: PITR do banco gerenciado + cópia imutável em outra região)	automático	abaixo

Rotina	Frequência	Como	Responsável	Evidência
Teste de restauração	mensal	<code>scripts/restore_test.sh</code> num banco descartável	dev de plantão	abaixo
Rotação de chaves	semestral ou em incidente	<code>FERNET_KEYS</code> + <code>rotate_keys</code> ; JWT por <code>kid</code> ; HMAC no cofre; certificados de veículo vencem em 90 dias	líder de segurança	teste de rotação
Revisão do threat model	trimestral e a cada feature nova	checklist STRIDE do PR + §4.1	time	<code>pull_request_template.md</code>

Auditoria de permissões executada (`rotinas.txt`):

	Usuário		Papel		Último login		Sessões ativas		403 no período		Situação	
	admin@ford.com		admin		2026-09-26		1		0		ok	
	analista@ford.com		analista		2026-09-26		1		0		ok	
	consultor@ford.com		consultor		2026-09-26		0		18		tentou acesso fora do papel	
Administradores: 1 (limite 3).												

Backup e restauração executados:

backup ok: backups/previopls-20260926T122611Z.dump.enc (48K)
00000000: 5361 6c74 6564 5f5f ... Salted___... ← arquivo cifrado, nada em claro
backups/previopls-20260926T122611Z.dump.enc: OK ← sha256 conferido
usuarios: producao=3 restaurado=3
clientes: producao=30 restaurado=30
leads: producao=8 restaurado=8
audit_logs: producao=84 restaurado=84
telemetria: producao=17 restaurado=17
restauracao ok em restore_teste_1790425572 (removido ao sair)

RPO de 24 h e RTO de 4 h no ambiente de demo; em produção, PITR de 5 min do banco gerenciado.

4.6. Checklist de conformidade

#	Item	Status	Evidência
1	Pipeline com SAST, SCA, secret scanning e container scanning	● OK	<code>.github/workflows/devsecops.yml</code>
2	Gates que quebram o build	● OK	<code>--error, --strict, exit-code: 1</code>
3	Actions pinadas por SHA e permissões mínimas	● OK	workflow
4	SBOM e assinatura de imagem	● OK	jobs 6 e 8
5	Nenhum segredo no repositório nem no histórico	● OK	gitleaks 0 (com triagem)
6	Nenhuma dependência com CVE conhecida (API de segurança)	● OK	pip-audit 0
7	Imagem sem HIGH/CRITICAL corrigível	● OK	trivy image
8	IaC sem misconfig HIGH/CRITICAL	● OK	trivy config, hadolint
9	PII cifrada em repouso e mascarada em resposta/log	● OK	§2.2, testes
10	Rotação de chave de PII testada	● OK	<code>test_rotacao...</code>
11	Rate limit, validação de entrada e JWT seguro	● OK	§2.3, testes
12	RBAC por perfil com matriz testada	● OK	<code>test_matriz_rbac</code>
13	MQTT com mTLS, ACL e sem texto claro	● OK	<code>iot-ataques.txt</code>
14	Containers não-root, read-only, sem capabilities	● OK	compose, k8s
15	Logs estruturados com correlação e sem PII	● OK	<code>logs-exemplo.txt</code>

#	Item	Status	Evidência
16	Métricas e alertas para API, IoT e ML	● OK	17 regras
17	Dashboards de segurança e operação	● OK	prints \$3.4
18	Plano de resposta com runbooks e exercício	● OK	\$3.5, runbooks/
19	Backup cifrado e restauração testada	● OK	rotinas.txt
20	Auditoria de permissões	● OK	rotinas.txt
21	Inventário LGPD, retenção e anonimização	● OK	\$4.4
22	MFA para admin/analista	● Parcial	roadmap
23	Escopo de lead por concessionária (BOLA)	● Residual	roadmap
24	Pinning e cache cifrado no app	● Parcial	roadmap mobile
25	Chaves em KMS/HSM e mTLS entre serviços internos	● Parcial	roadmap de produção
26	Segredo, SAST e SCA rodados nas outras frentes (SOA, mobile, IA/ML)	● OK	\$1.5
27	Achados F1–F4 das outras frentes corrigidos	● Residual	com cada disciplina

4.7. Riscos residuais e roadmap

Risco residual	Por que ainda existe	Quando resolver
BOLA entre concessionárias	o modelo de domínio ainda não associa consultor a concessionária	antes do piloto com mais de uma concessionária
Sem MFA	fora do escopo da sprint	antes de produção (admin e analista primeiro)
Rate limit e contadores de alerta em memória	uma réplica na demo	Redis (RATE_LIMIT_STORAGE_URI) ao escalar
Chave Fernet/JWT em variável/arquivo	ambiente acadêmico	Azure Key Vault / AWS KMS no deploy
Tráfego interno em HTTP dentro da rede do cluster	rede isolada	service mesh com mTLS (Linkerd/Istio)
Usuário único do banco escreve em audit_logs	migrations e app compartilham o papel	papéis separados (app_rw sem UPDATE/DELETE em audit_logs)
F1 e F2 no SOA (segredo padrão, Spring Boot com CVE crítica)	repositório de outra disciplina, entregue antes deste scan	antes de qualquer deploy do SOA
F3 e F4 (modelo sem hash; cleartext e cache no app)	idem	Sprint 4
ZAP só em modo baseline (passivo)	o scan ativo demora mais que o job e grava dados de teste	full scan autenticado no staging antes do piloto

Anexo A — histórico de commits

```

acead24 chore: baseline da sprint 2 (api de segurança FastAPI + STRIDE)
d370452 fix(auth): logout nunca disparava e falhas de login sumiam da trilha
31f0618 fix(ratelimit): limite global não era aplicado e X-Forwarded-For era forjável
d03e3d8 feat(auth): refresh reutilizado derruba a família, logout invalida o access e kill switch de sessão
b5c674c fix(deps): atualiza dependências apontadas pelo pip-audit (27 CVEs)
ac74137 fix(logging): máscara de CPF vazava 3 dígitos, segredo por nome de campo e X-Request-Id validado
71c519a fix(hmac): assinatura válida podia ser reenviada durante a janela de 5 min
777aede test: integração contra postgres cobrindo logout, RBAC, sessão, rate limit, XFF e HMAC
dad3069 feat(crypto): rotação de chave da PII com MultiFernet e boot recusa segredo fraco em produção
5d5c15e feat(obs): métricas prometheus da api e log JSON em arquivo pro loki
79c8939 feat(iot): telemetria de veículo conectado por MQTT com mTLS, ACL por VIN e validação no ingestor
4aee75d build(infra): imagem e compose endurecidos, nginx sem root e manifests k8s
8798e64 feat(obs): prometheus, alertmanager, loki, promtail e grafana com alertas de segurança
00464a8 ci: pipeline DevSecOps com secrets, SAST, SCA, IaC, testes, imagem, DAST e assinatura
fbc7418 feat(ops): auditoria de permissões, backup cifrado e teste de restauração
62d2dae fix(build): imagem sem pip em runtime (trivy HIGH) e warnings do hadolint
3c018ca fix(errors): 404/405 e erros da borda no mesmo contrato JSON
57b122c fix(obs): alerta da primeira ocorrência nunca disparava
31c1443 chore(secrets): triagem do gitleaks e demo mais robusta
ee289d2 chore(sast): login e refresh marcados como públicos de propósito na regra do semgrep
fef3568 docs: documento da sprint 3, diagramas, runbooks e evidências de execução
f4de7fc docs: evidências sem caminho local da máquina e pdf regenerado
a936b83 ci: corrige o workflow depois da primeira execução no github
b855fee docs: registra a primeira execução do pipeline no github
db4da88 ci: gate do trivy separado do relatório SARIF e banco de CVE por espelho
cb29e3e ci: DAST sobe a stack em runner limpo e não imprime senha no log
8452a90 docs: documento em A4 (Word + PDF) com capa, sumário e diagramas legíveis
fb18eb9 fix(infra): tag do mosquitto que existe no docker hub (2.1.2-alpine)
638fb66 fix(edge): headers de segurança uma vez só na resposta (ZAP 10035 no DAST)

```

`git show <hash>` mostra o diff de cada correção.

Anexo B — como reproduzir

```
python scripts/bootstrap_env.py           # segredos aleatórios em .env e .secrets/
docker compose up -d --build              # 12 serviços
docker compose --profile demo run --rm trafego # uso normal + ataques na API (~6 min)
docker compose --profile demo run --rm simulador --modo ca-falsa # e os outros modos de IoT
```

Endereço	O quê
https://localhost:8443	API pela borda (a CA de dev fica no volume cyber3_certs_nginx)
mqtt://localhost:8883	broker MQTT (mTLS)
http://127.0.0.1:3300	Grafana (admin / senha impressa pelo bootstrap_env.py)
http://127.0.0.1:9090	Prometheus
http://127.0.0.1:9093	Alertmanager

Testes: TEST_DATABASE_URL=postgresql+psycopg://... python -m pytest -q (sem banco, só os unitários rodaram).